ISO/IEC JTC1 SC22 WG21 P2491R0
Author: Jens Maurer
Target audience: SG16, LEWG
2021-11-15

P2491R0: Text encodings follow-up

1 Abstract

This paper discusses some design decisions of P1885 "Naming Text Encodings to Demystify Them" by Corentin Jabot and Peter Brett that, in the view of the author, are going in the wrong direction for a number of seemingly small, but crucial design decisions.

In short,

- P1885 re-uses the encoding "UTF16" for a different purpose, which causes confusion.
- P1885 cannot properly represent the UCS-2 wide encoding used on Microsoft Windows before the switch to UTF-16
- P1885 attempts to specify object representation, which breaks an important C++ abstraction barrier

2 Paper history

R0: initial revision

3 Use cases for the `std::text_encoding` facility

- Describe the encoding of text transmitted over the network or stored in a file.
- Describe the encoding for C++ ordinary and wide string literals.
- Describe the encoding of the C++ environment (e.g. environment variables or the console).
- Facilitate conversion between encodings (an actual conversion facility is not in scope).

In all of these cases, the character set (i.e. the set of individual characters) supported in a specific context is indirectly defined by the encoding, but not explicitly specified.

4 Building blocks

Text encodings are used in a variety of situations:

- Identifying the encoding of text files or network streams
- Identifying the C++ ordinary and wide literal encodings
- Identifying the environment (e.g. terminal) encoding

4.1 C++ ordinary and wide literal encodings

[lex.charset] specifies in the current working draft:

> A *code unit* is an integer value of character type (6.8.2). Characters in a *character-literal* [...] or in a *string-literal* are encoded as a sequence of one or more code units [...]; this is termed the respective

> literal encoding. The *ordinary literal encoding* is the encoding applied to an ordinary character or string literal. The *wide literal encoding* is the encoding applied to a wide character or string literal.

Then, [lex.string] p10.1 specifies

> The sequence of characters denoted by each contiguous sequence of *basic-s-char*s, *r-char*s, *simple-escape-sequence*s (5.13.3), and *universal-character-name*s (5.3) is encoded to a code unit sequence using the *string-literal*'s associated character encoding.

Thus, an encoding for ordinary and wide literals in C++ relates a sequence of characters with a sequence of integer values of the respective character type (`char` or `wchar_t`).

4.2 IANA list of character sets

The IANA (Internet Assigned Numbers Authority) maintains a registry of encodings (called "character sets") at https://www.iana.org/assignments/character-sets/character-sets.xhtml, as instigated by RFC2978.

As described by RFC2978:

> The term "charset" (referred to as a "character set" in previous versions of this document) is used here to refer to a method of converting a sequence of octets into a sequence of characters.

Thus, an encoding in the IANA registry relates a sequence of octets with a sequence of characters.

4.3 Unicode

Unicode provides the concept of an encoding form for the relationship between a sequence of characters (specifically, a sequence of code points) and a sequence of integer values. Unicode further provides the concept of an encoding scheme for the relationship between a sequence of characters and a sequence of octets.

Regrettably, the specified encoding forms and encoding schemes have overlapping naming; "UTF-16" refers both to an encoding form and an encoding scheme.

UTF-8

ISO 10646:2020 section 10.2 specifies the encoding form as follows:

> UTF-8 is the UCS encoding form that assigns each UCS scalar value to an octet sequence of one to four octets, as specified in table 2.

The encoding scheme is defined as follows in section 11.2:

> The UTF-8 encoding scheme serializes a UTF-8 code unit sequence in exactly the same order as the code unit sequence itself.

Thus, for UTF-8, the code units are octets and those octets also constitute the encoding scheme. This encoding does not depend on endianness (byte order in the object representation of an integer) at all.

UTF-16

ISO 10646:2020 section 10.3 specifies the encoding form called "UTF-16" as follows:

> UTF-16 is the UCS encoding form that assigns each UCS scalar value to a sequence of one to two unsigned 16-bit code units, as specified in table 4.

The encoding scheme called "UTF-16" is specified in section 11.5 as follows:

> The UTF-16 encoding scheme serializes a UTF-16 code unit sequence by ordering octets in a way that either the less significant octet precedes or follows the more significant octet. In the UTF-16 encoding scheme, the initial signature read as <FE FF> indicates that the more significant octet precedes the less significant octet, and <FF FE> the reverse. The signature is not part of the textual data. In the absence of signature, the octet order of the UTF-16 encoding scheme is that the more significant octet precedes the less significant octet.

The "initial signature" is otherwise known as a byte order mark (BOM).

The Unicode standard version 14.0.0 specifies in section 3.10:

> UTF-16 encoding scheme: The Unicode encoding scheme that serializes a UTF-16 code unit sequence as a byte sequence in either big-endian or little-endian format.
>
> [...]
>
> In the UTF-16 encoding scheme, an initial byte sequence corresponding to U+FEFF is interpreted as a byte order mark; it is used to distinguish between the two byte orders. An initial byte sequence <FE FF> indicates big-endian order, and an initial byte sequence <FF FE> indicates little-endian order. The BOM is not considered part of the content of the text.
>
> The UTF-16 encoding scheme may or may not begin with a BOM. However, when there is no BOM, and in the absence of a higher-level protocol, the byte order of the UTF-16 encoding scheme is big-endian.

Note the caveat of an undefined "higher-level protocol", which does not exist in ISO 10646.

In either standard, there are also encoding schemes UTF-16LE and UTF-16BE that do not interpret a signature (byte order mark) at all, but use the given big-endian or little-endian layout unconditionally.

UTF-32

The specifiation of UTF-32 is analogous to UTF-16. There is no provision for endianness other than big-endian or little-endian.

4.4 iconv

POSIX

`iconv` is a transcoding function specified by POSIX:

```
size_t iconv(iconv_t cd, char **restrict inbuf,
       size_t *restrict inbytesleft, char **restrict outbuf,
       size_t *restrict outbytesleft);
```

The conversion descriptor (the first argument) is created using `iconv_open`:

```
iconv_t iconv_open(const char *tocode, const char *fromcode);
```

with the following specification:

> The `iconv_open()` function shall return a conversion descriptor that describes a conversion from the codeset specified by the string pointed to by the `fromcode` argument to the codeset specified by the string pointed to by the `tocode` argument. [...]

> Settings of `fromcode` and `tocode` and their permitted combinations are implementation-defined.

As a non-normative note, `iconv` says:

> The objects indirectly pointed to by `inbuf` and `outbuf` are not restricted to containing data that is directly representable in the ISO C standard language `char` data type. The type of `inbuf` and `outbuf`, `char **`, does not imply that the objects pointed to are interpreted as null-terminated C strings or arrays of characters. Any interpretation interpretation of a byte sequence that represents a character in a given character set encoding scheme is done internally within the codeset converters. For example, the area pointed to indirectly by inbuf and/or outbuf can contain all zero octets that are not interpreted as string terminators but as coded character data according to the respective codeset encoding scheme. The type of the data (`char`, `short`, `long`, and so on) read or stored in the objects is not specified, but may be inferred for both the input and output data by the converters determined by the `fromcode` and `tocode` arguments of `iconv_open()`.

Thus,

- A codeset value is understood to implicitly specify the object type for the character data, which may be different from `char`.
- There is no normative requirement which codesets must be supported.

GNU iconv

GNU iconv implements POSIX iconv as follows:

- It recognizes some of the dangers involved with the type-unsafety of a `char*` parameter possibly pointing to objects of other integer types.
- The implementaton of reading the "UTF-16" codeset uses the platform endianness as the default in absence of a byte order mark, which conforms to the Unicode encoding scheme "UTF-16" assuming that the higher-level protocol is the platform endianness. However, that implementation does not conform to the ISO 10646 encoding scheme "UTF-16", which requires big-endian encoding in the absence of a byte order mark.
- The implementation of writing the "UTF-16" codeset always writes a byte order mark.

4.5 ICU

ICU also comes with an encoding converter; the list of supported aliases is here.

5 Special handling for UTF-16 and UTF-32

As described above, UTF-16 as an encoding scheme has the following different interpretations:

- ISO 10646: BOM-aware, big-endian if absent
- Unicode 14: BOM-aware, higher-level protocol if absent, default big-endian
- IANA: BOM-aware, recommendation for big-endian if absent (see RFC2781)
- iconv: reading: BOM-aware, platform endianness if absent; writing: always writes a BOM
- C++ wide literal on Windows: wchar_t contains UTF-16 code units; encoding scheme is identical to UTF16LE

P1885 elects to map the correct UTF16LE/BE encoding scheme identifier possibly returned from the `std::text_encoding::wide_literal()` function to UTF16. This is user-unfriendly for the following reasons:

- There is no differentiation of the encoding label between text that has arrived from the network and is tagged "UTF16" in the IANA sense (objects of type `char`, a BOM is expected to be present) vs. the

UTF16LE/BE text that is produced from a wide literal (objects of type `wchar_t`, without a BOM).
- `iconv` always creates a BOM when writing the UTF-16 encoding. If a user were to convert third-party text from e.g. UTF-8 to "UTF16" for use with `std::wstring` and string literals, BOMs are likely to end up in the middle of a string.

Futher, `iconv`, presumably one of the premier consumers of the object representation model (see below), was designed with the understanding that the encoding name also conveys the object type for each code unit (e.g. `char` or `int` or, presumably, `wchar_t`). This distinction is lost when both network data (in a `char` buffer) and `wchar_t` literals are expected to be described with the same `std::text_encoding` value.

It is conceivable to introduce a new enumerator `UTF16NE` that has the value of either of the existing enumerators `UTF16LE` or `UTF16BE` (as appropriate) and return that value from `std::text_encoding::wide_literal()` on e.g. Windows platforms. This approach, as well as an earlier approach in P1885 that returns either UTF16LE or UTF16BE, but never UTF16, would redundantly represent information about platform endianness in an unrelated part of the standard. Platform endianness should be handled exclusively by the existing targeted facility `std::endian` (see 26.5.8 [bit.endian]).

P1885 also elects to map UTF16LE/BE to UTF16 for the non-wide `std::text_encoding::literal()`. Since `CHAR_BIT == 8` is required for this function, the ordinary literal encoding can never be UTF-16. If it were, two consecutive `char` elements would be used to represent a single code unit, but some `char` elements might have the value 0 without representing the null character. This is not a valid encoding per [lex.charset]. The mapping is thus superfluous for the result of `std::text_encoding::literal()`.

Everything said above also applies analogously to UTF-32.

6 No special handling for UCS2

UCS-2 was effectively used on the Microsoft Windows (little-endian) platform for a decade or so before it was switched to UTF-16.

The usage situation is approximately the same as that for UTF-16, yet P1885 does not even attempt to perform any mapping that could be viewed as removing endianness assumptions from the name. Adding to that, the IANA registry appears to define "UCS2" as big-endian, but does not make any allowance for a little-endian UCS-2 encoding scheme. This leaves the relevant (admittedly outdated) Microsoft Windows platforms conceptually unsupported.

7 Looking at the object representation breaks an abstraction barrier

The C++ object model carefully avoids considering the object representation. Where it must do so (e.g. for `bit_cast`), lots of care needs to be applied to properly deal with padding bits, partially uninitialized values, and other obscure situations.

I believe it is a mistake that P1885 talks about specifying the object representation by applying an encoding scheme. The object representation should never be in the focus of a user or the specification of a user-facing facility.

The following alternative model avoids talking about the object representation, naturally supports implementations with `CHAR_BIT > 8` or with `sizeof(wchar_t) == 1`, and allows proper differentiation between literal encodings and network data.

- The IANA encoding registry is understood to list encoding schemes, i.e. octet-based encodings.
- An octet as provided by an IANA encoding is mapped to a single element of a string (i.e. a value of type `char` or `wchar_t`); each octet value of an IANA encoding is thus understood to be a code unit.

- In addition to the IANA list, new encodings with new MIB values outside of the IANA-controlled number space are introduced to represent popular wide literal encodings, namely "WIDE.UTF16", "WIDE.UTF32", "WIDE.UCS2", and "WIDE.UCS4".

Observations:

- Per [intro.memory], a byte (equivalently, a `char`) is at least 8 bits and thus can hold the value of an octet.
- GNU `iconv` (and likely other implementations) does not currently support the "WIDE.*" names. This can reasonably be expected to change when the names are standardized.
- Some encodings do not make sense in certain situations. For example, "UTF-8" is unlikely to make sense for a `std::wstring` if `sizeof(wchar_t) > 1`.
- IANA-registered encodings can possibly be used for `wchar_t` strings if `sizeof(wchar_t) == 1`.
- The "WIDE.*" encodings can possibly be used for `char` strings if `CHAR_BIT >= 16`. There is no difference regarding the string literal encoding approach between a `char` with 16 bits and a `wchar_t` with 16 bits, regardless of whether the latter consists of one or two bytes.
- No endianess information is conveyed by the "WIDE.*" encodings. This probably makes them unsuitable to describe a network transmission, but allows to properly separate the concerns of platform endianness from those of the code unit representation of string literals. (Historically, there have been platforms that are neither big-endian nor little-endian.)

This proposal does not limit the choice of encoding for the platform, but does allow to express all reasonable encodings even for fringe (but valid) abstract machine parameters such as `CHAR_BIT >= 16` or `sizeof(wchar_t) == 1`.

8 Wording plan

Relative to P1885, the wording should be adjusted as follows:

- Specify that the octets of the encoding schemes in the IANA registry are considered as code units for purposes of `std::text_encoding`.
- Specify additional encodings "WIDE.UTF16", "WIDE.UTF32", "WIDE.UCS2", and "WIDE.UCS4" with negative enumerator values to avoid conflict with present or future IANA assignments.
- Add a note that IANA encoding schemes cannot be returned from `std::text_encoding::(wide_)literal()` unless `sizeof(char_type) == 1`.
- Remove restrictions about `CHAR_BIT == 8` or `sizeof(wchar_t) > 1` (if any).
- Adjust existing conflicting wording as appropriate.